

Electronic System Level Design Methodology

Project 2, 2026-05-14

Abstract

Use a RISC-V or ARMv8 processor with a YOLOv5l co-processor to perform object detection for traffic applications. The system is modeled using Synopsys Virtualizer.

Please read carefully. All outputs required are described in the text. Five (5) points will be taken for each missing required output and behavior.

Overview

In this project, you will explore the design and implementation of a traffic object detection embedded system using **SystemC**. The system consists of a **RISC-V** or **ARMv8** processor and a YOLOv5l (YOLOv5 large) co-processor, connected via buses to local and shared memories, and is capable of processing and analyzing image data containing traffic-related objects such as humans, road signs, lane marks, and various types of vehicles. You will need to determine the cache structures and sizes of the processor and co-processor, as well as the sizes of local and shared memories. Also, to determine whether the DMA and/or DMA2 of Assignment 1 and Project 1 should be used. And then design the system memory map and architecture. This lab combines concepts from embedded systems, distributed processing, memory sharing, and machine learning and inference.

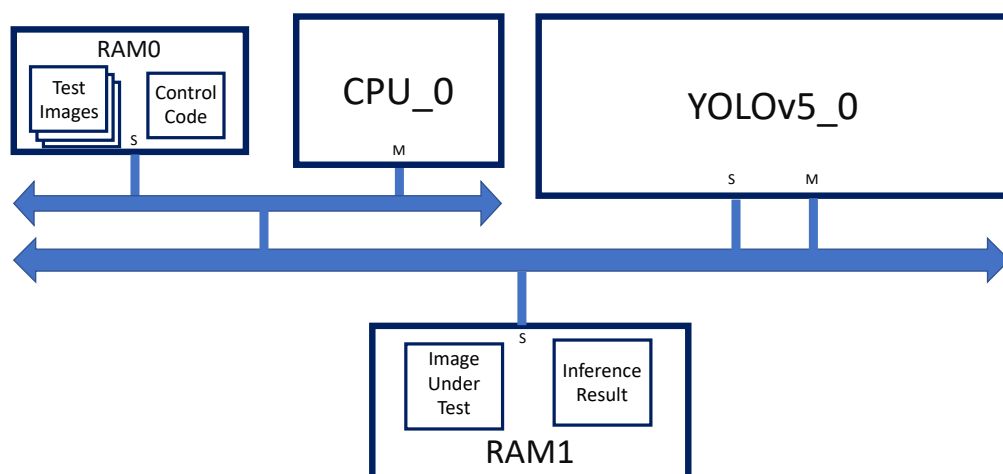
System Architecture

The system is composed of the following components:

- **CPU_0 (Image Reader and Renderer)**
 - A **RISC-V** or **ARMv8** processor
 - Loads input test images containing traffic objects.
 - Passes image data to shared memory to be processed by the YOLOv5l component.
 - Receives detection results and outputs bounding boxes in text, i.e., origin x and y coordinates and width and length.
- **YOLOv5_0 (Traffic Object Detection Unit)**

- Import a trained YOLOv5l model as a Virtualizer model and the co-processor for embedded inference.
 - Reads image data from the shared memory.
 - Detects traffic objects and returns bounding box coordinates to CPU_0.
- **RAM0, The Local Memory**
 - The CPU_0 core has its own cache and local memory (RAM0) for independent processing. Through experiments, you will determine the cache structure and size, as well as the RAM0 size, based on the code and data sizes.
- **RAM1, The Shared Memory**
 - Used as a communication buffer to exchange image data and detection results between the processor and the co-processor. You will also need to determine the shared memory size based on the input image data and the returned inference results, the bounding boxes, so to speak.
- **DMA/DMA2**
 - Please decide whether you would like to reuse DMA and/or DMA2 of Assignment 1 and Project 1.
- **Memory Map**
 - To determine the sizes of local and shared memory, you need to design the target system's memory map. Do include the address of the DMA/DMA2 in the memory map, should you decide to use it in your system. **You must include the memory map of your design in the 5-page report.**

The system is illustrated in the following block diagram:



Notice that DMA/DMA2 are not included because it is up to you to decide whether they should be included or not.

Deliverables

- Write a less than 5-page report to describe the SoC memory map and architecture, and how you train the YOLOv5l and port it into the system.
- Submit your work to Moodle before the project deadline and run a demo in the SoC Lab. In the demo you need to demonstrate your work using Virtualizer GUI to view RISC-V/ARMv8 code execution and bus transactions.

Due Date

15:00, June 27th, 2026, at the SoC Lab.

Score weight (towards the final grade) 40%

- 10%: Submit the less than 5-page report
- 5%: Complete the YOLOv5l training
- 5%: Complete the SoC memory map and the architecture in Virtualizer
- 20%
 - 10% to complete CPU0 bare metal program and run
 - 5% for the CPU0 to successfully pass three (3) test images to YOLOv5l and receive the inference results.
 - 5% for the YOLOv5l inference model to successfully detect traffic objects.

The following outlines the suggested steps to execute this project.

YOLOv51 Training

Training YOLOv51 with KITTI Object Detection & 3D Benchmark on the **NVIDIA RTX 4090** via [Glows.ai](https://glows.ai) takes about 1~2 hours of computing time. The 4090's 24GB of VRAM allows for larger batch sizes, which stabilize training and significantly speed up the process. Here is the YOLOv51 configuration and training guide:

Step 1. Environment Setup (Glows.ai Instance)

When creating an instance on Glows.ai, one should select a **PyTorch** image. These images usually come pre-installed with CUDA and the necessary drivers. When you configure a virtual machine on Glows.ai, please select the YOLOv5 container of `img-9rp4vyp3`. It contains Ubuntu 20.04, CUDA 12.4, PyTorch 2.6.0, Python 3.11, JupyterLab/VNC, YOLOv5, and ultralytics 8.3.92. You can also run the following command on your own Linux machine.

```
# Clone the YOLOv5 repository and install dependencies
git clone https://github.com
cd yolov5
pip install -r requirements.txt
```

Step 2. Download and Organize KITTI Data

Since the KITTI dataset is hosted by the Karlsruhe Institute of Technology, students should use `curl` or `wget` to download the files directly to the Glows.ai instance to save time and bandwidth.

– Create the Directory Structure

Execute the following commands on the Glows.ai virtual machine.

```
# Navigate to the parent directory of yolov5
cd ..
mkdir -p datasets/kitti/training
cd datasets/kitti
```

– Download the Required Files

You will need the left color images and the training labels.

Note: The following URLs are placeholders for the official KITTI download links which usually require a registered email. You should use the links provided in their account.

```
# Download Left Color Images (12 GB)
wget -O data_object_image_2.zip "https://amazonaws.com"
```

```
# Download Training Labels (5 MB)
wget -O data_object_label_2.zip "https://amazonaws.com"
```

– Extract the Files

Next, unzip the images and labels.

```
unzip data_object_image_2.zip -d .
unzip data_object_label_2.zip -d .

# Verify the structure
# It should look like:
# datasets/kitti/training/image_2/
# datasets/kitti/training/label_2/
```

– Cleanup

To save disk space on the Glows.ai instance, remove the extracted `.zip` files.

```
rm *.zip
```

Step 3. Dataset Format Conversion (KITTI to YOLO)

KITTI's native labels use absolute pixel coordinates (e.g., `Car 0.00 0 ... 712.40 143.00 810.73 307.92`). YOLOv5 requires normalized relative center coordinates.

- **Action Item:** You must convert KITTI labels to `.txt` files where each line follows: `<class_id> <x_center> <y_center> <width> <height>`.
- **Class Mapping:** Since the project focuses on traffic detection, it is recommended to map the 8 KITTI classes into 3 main categories: `Car`, `Pedestrian`, and `Cyclist`.

The following script will help you convert the original KITTI labels into the normalized format required by YOLOv5. You should save this as `convert.py` in your working directory. It uses OpenCV to get image dimensions and performs the necessary normalization. Install OpenCV with `pip install opencv-python`. And run the script with `python convert.py`.

```
import os
import cv2

def convert_kitti_to_yolo(img_dir, label_dir, output_dir, classes=['Car',
'Pedestrian', 'Cyclist']):
```

```

"""
Converts KITTI label format to YOLO label format.
KITTI: cls truncated occluded alpha x1 y1 x2 y2 ...
YOLO:  cls_id x_center y_center width height (all normalized 0-1)
"""

if not os.path.exists(output_dir):
    os.makedirs(output_dir)

for label_file in os.listdir(label_dir):
    if not label_file.endswith('.txt'):
        continue

    with open(os.path.join(label_dir, label_file), 'r') as f:
        lines = f.readlines()

    # Image is needed to get width/height for normalization
    img_path = os.path.join(img_dir, label_file.replace('.txt', '.png'))
    img = cv2.imread(img_path)
    if img is None:
        print(f"Warning: Image {img_path} not found. Skipping.")
        continue

    h, w, _ = img.shape
    yolo_labels = []

    for line in lines:
        parts = line.split()
        cls = parts[0]

        # Filter only desired classes for the traffic project
        if cls not in classes:
            continue

        cls_id = classes.index(cls)

        # KITTI BBox coordinates (x1, y1, x2, y2) are in parts[4:8]
        x1, y1, x2, y2 = float(parts[4]), float(parts[5]),
float(parts[6]), float(parts[7])

```

```

        # Calculate YOLO format
        xc = (x1 + x2) / 2 / w
        yc = (y1 + y2) / 2 / h
        nw = (x2 - x1) / w
        nh = (y2 - y1) / h

        # Ensure values stay within [0, 1] range
        yolo_labels.append(f"{cls_id} {max(0, min(1, xc)):.6f} {max(0, min(1, yc)):.6f} {max(0, min(1, nw)):.6f} {max(0, min(1, nh)):.6f}")

    # Save the new label file
    with open(os.path.join(output_dir, label_file), 'w') as f:
        f.write('\n'.join(yolo_labels))

    print(f"Conversion finished! Labels are in: {output_dir}")

# --- Instructions for Users ---
# 1. Update the paths below to match your Glows.ai folder structure.
# 2. Run the script: python convert.py
if __name__ == "__main__":
    TRAIN_IMG = "datasets/kitti/training/image_2"
    TRAIN_LBL = "datasets/kitti/training/label_2"
    OUTPUT_LBL = "datasets/kitti/training/labels" # YOLOv5 looks for
'labels' folder

    convert_kitti_to_yolo(TRAIN_IMG, TRAIN_LBL, OUTPUT_LBL)

```

Step 4. Dataset Splitting Script

For a machine learning project, a proper Train/Validation/Test split is critical to ensure you aren't "cheating" by testing on data the model has already seen. Since the KITTI Object Detection benchmark officially provides only a "Training" set (with labels) and a "Testing" set (without labels), you must split the official training set yourself. It is recommended a 80/20 split (80% Training, 20% Validation).

The following script will organize the converted data into the folder structure expected by YOLOv5.

```

import os
import random
import shutil

def split_kitti(root_dir, train_ratio=0.8):
    # Paths based on previous conversion step
    img_dir = os.path.join(root_dir, 'image_2')
    lbl_dir = os.path.join(root_dir, 'labels')

    # Create YOLO structure
    base_out = 'datasets/kitti_final'
    for folder in ['images/train', 'images/val', 'labels/train',
'labels/val']:
        os.makedirs(os.path.join(base_out, folder), exist_ok=True)

    # Get all file stems (e.g., '000001')
    all_stems = [f.split('.')[0] for f in os.listdir(lbl_dir) if
f.endswith('.txt')]
    random.shuffle(all_stems)

    split_idx = int(len(all_stems) * train_ratio)
    train_stems = all_stems[:split_idx]
    val_stems = all_stems[split_idx:]

    def move_files(stems, subset):
        for stem in stems:
            # Copy Image
            shutil.copy(os.path.join(img_dir, f"{stem}.png"),
                        os.path.join(base_out,
f"images/{subset}/{stem}.png"))
            # Copy Label
            shutil.copy(os.path.join(lbl_dir, f"{stem}.txt"),
                        os.path.join(base_out,
f"labels/{subset}/{stem}.txt"))

    print(f"Splitting {len(all_stems)} images...")
    move_files(train_stems, 'train')
    move_files(val_stems, 'val')

```

```

    print(f"Done! Train: {len(train_stems)}, Val: {len(val_stems)}")

if __name__ == "__main__":
    # Point this to the KITTI training folder
    split_kitti('datasets/kitti/training')

```

Step 5. Create the `kitti.yaml` Configuration

First, please create a file named `kitti.yaml` inside the `yolov5/data/` directory and copy the following code into `kitti.yaml`:

```

# 1. Dataset root directory
# Move the dataset kitti_final prepared earlier, into ../dataset
# It is recommended to use an absolute path, or a path relative to
# the train.py script
path: ../dataset/kitti_final # Dataset root directory

# 2. Training and Validation image paths (relative to 'path')
train: images/train          # Path to training images
val: images/val              # Path to validation images

# Number of classes and their names
nc: 3

# 4. Class names
# The order must exactly match the index used in your conversion
# script.
names: ['Car', 'Pedestrian', 'Cyclist']

```

Key Requirements:

- **Path Consistency:** The `path` variable must point to the folder created by the **Dataset Splitting Script** provided earlier (`kitti_final`).
- **Class Indexing:** The order in the `names` list is crucial. If `Car` is index 0, then all converted `.txt` label files must use 0 as the class ID for cars.
- **Directory Structure:** Ensure the `kitti_final` folder follows this standard YOLO structure:
 - `kitti_final/images/train/`
 - `kitti_final/labels/train/`
 - `kitti_final/images/val/`

- o kitti_final/labels/val/

Step 6. Training Command and RTX 4090 Optimization

On Glows.ai, you can select from TW-02, TW-03, and TW-04 to construct VMs. However, training GPUs are available only in TW-02 and TW-04, with TW-02 having the powerful and expensive H100. And in TW-03 there are inference GPUs. And in all the three TW regions I have created a team drive for each for you to use KITTI object detection image and label files. When you create a new VM instance, after selecting the GPU you want to use and the YOLOv5 ultralytics 8.3.82 model, make sure to click the **Mount Team Datadrive** button to select the **datadrive**.

When the VM is up and running, the **Team Datadrive** is at `/team_data`. In it you can find `datasets/kitti/training` and `datasets/kitti/testing`. In the training folder, you can find 3 folders: `image_2`, `label_2`, and `labels`. Ignore the `label_2` folder, as it uses the original KITTI label format. I converted these label files into YOLO format in `labels`.

The YOLOv5 folder is at `/yolov5`. You need to modify the `train.py` to use the training dataset as described above. And you do not have write access to `/team_data`, so you need to store the trained model in the VM's home directory. The VM comes with its own sizable storage space for at least 100GB, depending on which GPU you choose. The biggest is 500GB for H100. Remember to download the trained YOLOv5l model before you stop and release the VM, for it is lost after the VM is stopped and released.

In the 4090 virtual machine, you can run the command below to enable larger batch sizes and Automatic Mixed Precision (`--amp`) to maximize efficiency. The `-weights yolov5l.pt` flag sets the creation of a YOLOv5 large model.

```
python train.py \  
  --img 640 \  
  --batch 32 \  
  --epochs 100 \  
  --data data/kitti.yaml \  
  --weights yolov5l.pt \  
  --device 0 \  
  --amp \  
  --workers 8
```

Parameter Breakdown:

- `--img 640` : Standard resolution. If you wish to improve detection of distant vehicles, you can try `--img 1280` , but the `batch` size should be lowered to 16 to fit VRAM.
- `--batch 32` : The 4090's 24GB VRAM handles YOLOv5l easily at this size. Larger batches provide more stable gradients for the BatchNorm layers.
- `--amp` : Enables Automatic Mixed Precision. This speeds up training and reduces VRAM usage without a significant loss in accuracy.
- `--workers 8` : This utilizes the CPU cores of the Glows.ai instance to pre-load data, ensuring the GPU is never "starved" of images.
- `--weights yolov5l.pt` : This uses the pre-trained YOLOv5 model to train on the KITTI dataset and create the YOLOv5l model. Use `-weights yolov5m.pt` to create a medium model YOLOv5m and `-weights yolov5s.pt` for a small model YOLOv5s.

Project Management Tips

Given the 40-hour limit, you should follow these best practices:

- **Use Checkpoints:** YOLOv5 automatically saves `last.pt` and `best.pt` . If a session times out, you can resume using `--resume` .
- **Validation:** You should check the `results.png` file generated in the `runs/train/exp/` folder to monitor mAP (mean Average Precision) and loss curves.
- **Data Persistence:** Remember to download your `best.pt` weights or move them to a persistent [Glows.ai Data Drive](#) before the instance is terminated.

Usage Notes

- **Directory Structure:** YOLOv5 expects images and labels to be in the same parent directory, e.g., `images/train/1.png` and `labels/train/1.txt` .
- **Library Requirement:** You may need to install OpenCV using `pip install opencv-python` .
- **Validation:** It is recommended to use the ultralytics Hub or the built-in `val.py` script to verify that the conversion was successful before starting a long training run.

Import The YOLOv51 Model to a SystemC Module

To integrate your trained YOLOv51 model for a Virtualizer simulation, you must bridge the gap between the high-level PyTorch model and the transaction-level SystemC environment.

Step 1: Export the YOLOv51 Model to a C++ Compatible Format

The native `.pt` (PyTorch) format is difficult to use in a bare-metal or SystemC environment. You must export it to **ONNX** or **TorchScript**.

1. **Export to ONNX:** This is the most portable format for C++ inference. Note that the best `.pt` is created at the training.

```
python export.py --weights best.pt --include onnx --simplify --img 640
```

2. **Verify Compatibility:** Ensure the exported model can be loaded by a C++ inference engine like ONNX Runtime or OpenCV DNN.

To verify the compatibility of your exported YOLOv51 ONNX model within a C++ environment using ONNX Runtime (ORT), you should follow these steps below. This process ensures the model's inputs and outputs are correctly mapped before integrating it into a SystemC module.

Step 1. Step 1: Install ONNX Runtime C++

You need the ORT headers and binaries.

- **On Linux:** Download the ORT C++ Releases.
- **On Windows (Glows.ai):** Use `vcpkg install onnxruntime` or download the pre-built DLLs.

Step 2. Inspect Model Metadata (Sanity Check)

Before writing C++ code, use a tool like **Netron** (web version: netron.app) or a Python snippet to identify the exact input/output node names.

- **YOLOv5 standard input:** `images` (Shape: `1, 3, 640, 640`).
- **YOLOv5 standard output:** `output0` (Shape: `1, 25200, 8`).

Step 3. Minimal C++ Verification Script

Create a file named `verify_onnx.cpp` and copy the code below into it. This code initializes the ORT environment, loads the model, and checks if the engine can allocate memory for the specified tensors.

```
#include <iostream>
#include <vector>
#include <onnxruntime_cxx_api.h>

int main() {
    // 1. Initialize Environment
    Ort::Env env(ORT_LOGGING_LEVEL_WARNING, "YOLOv5_Verification");
    Ort::SessionOptions session_options;
    session_options.SetIntraOpNumThreads(1); // Keep it simple for SystemC
context

    // 2. Load the Model
    const char* model_path = "best.onnx";
    try {
        Ort::Session session(env, model_path, session_options);
        std::cout << "Successfully loaded: " << model_path << std::endl;

        // 3. Verify Input/Output counts
        size_t num_input_nodes = session.GetInputCount();
        size_t num_output_nodes = session.GetOutputCount();
        std::cout << "Inputs: " << num_input_nodes << " | Outputs: " <<
num_output_nodes << std::endl;

        // 4. Check Input Dimensions
        auto input_info = session.GetInputTypeInfo(0);
        auto input_tensor_info = input_info.GetTensorTypeAndShapeInfo();
        std::vector<int64_t> input_dims = input_tensor_info.GetShape();

        std::cout << "Input Shape: [";
        for (auto d : input_dims) std::cout << d << " ";
        std::cout << "]" << std::endl;

    } catch (const Ort::Exception& e) {
```

```

        std::cerr << "Error: " << e.what() << std::endl;

        return 1;
    }

    return 0;
}

```

Step 4. Compile and Link

You must link against the `onnxruntime` library. A sample `g++` command:

```

g++ verify_onnx.cpp -o verify_onnx \
    -I/path/to/onnxruntime/include \
    -L/path/to/onnxruntime/lib \
    -lonnxruntime -std=c++17

```

Step 5. Functional Validation (Data Flow)

Once the model loads, verify that the **C++ inference results** match the **Python results**.

1. **Preprocessing:** Ensure the image is converted to `float32`, scaled to `[0, 1]`, and transformed from HWC (OpenCV default) to **CHW** (YOLOv5 expectation).
2. **Run Inference:** Use `session.Run(...)`.
3. **Check Values:** Print the first 5 values of the output tensor. If they are identical (or within a tiny floating-point margin) to the values seen in Python, the model is compatible.

Common Troubleshooting for KITTI/YOLOv5

- **Dynamic vs. Static Axes:** If the model was exported with `--dynamic`, the batch size might show as `-1`. For **SystemC/Virtualizer**, it is safer to export with **static shapes** (`--img 640` and no dynamic flag).
- **Opset Version:** Use **Opset 12 or 13**. Newer versions (like 17+) might not be supported if the Virtualizer's underlying libraries are older. [12]

Step 6. Wrap YOLOv5L in a C++ Inference Class

Create a standalone C++ class (e.g., `YOLOv5Detector`) that handles model loading, image preprocessing (resizing to 640x640, normalization), and post-processing (Non-Maximum Suppression).

- **Header:** Define methods for `loadModel()`, `preprocess()`, and `detect()`.

- **Implementation:** Use ONNX Runtime C++ API or LibTorch to execute the model. This code will eventually be called by the SystemC module.

Step 7. Create the SystemC Module Wrapper

In SystemC, you will wrap your C++ detector inside an `sc_module`. This module acts as a "Hardware Abstraction" or a "Virtual Peripheral" that the Cortex-R52 interacts with via a bus (e.g., AHB or AXI).

Example SystemC Structure:

```
#include <systemc.h>
#include "YOLOv5Detector.h" // Your C++ Wrapper

SC_MODULE(YOLOv5_HW) {
    sc_in<bool> clk;
    // TLM-2.0 target socket for communication with Cortex-R52
    tlm_utils::simple_target_socket<YOLOv5_HW > socket;

    YOLOv5Detector* detector;

    SC_CTOR(YOLOv5_HW) : socket("socket") {
        detector = new YOLOv5Detector("best.onnx");
        socket.register_b_transport(this, &YOLOv5_HW::b_transport);
    }

    // Process memory-mapped transactions from the R52
    void b_transport(tlm::tlm_generic_payload& trans, sc_time& delay) {
        // 1. Read image data from 'trans' (simulated DMA or register write)
        // 2. Call detector->detect(image_data)
        // 3. Write results back to 'trans'
    }
};
```

Step 8. Import YOLOv5_HW Module into Virtualizer

Refer to Project 1 and import the YOLOv5_HW SystemC module into Virtualizer.

Workflow Summary

- **Task A:** Export `best.pt` to `best.onnx`.

- **Task B:** Write a C++ testbench to ensure the ONNX model runs correctly on their local machine.
- **Task C:** Integrate the C++ code into the SystemC template provided in the class.
- **Task D:** Write a small C program for the Cortex-R52 to "trigger" the detection and read back the bounding box results.